

Embeddings to Words Note-Taking Framework

Cole Drumheller

Friday 5th September, 2025 - 03:31

1 Summary

This week we learned about taking embeddings to words. We looked at models that are given these massive datasets of words with the goal of identifying words that are similar to one another. Let's start by talking about the youtube video, Vectoring Words (Word Embeddings) - Computerphile. In this video the guy described how neural networks go about processing the data and making correlations between similar words. The way that this is accomplished is by taking words and turning them into vectors that are representative of the context in which the words are used. The way he described it was that we have to assume that similar words will be used in similar contexts ((Vectoring Words (Word Embeddings) - Computerphile, 10:30-11:00). With this in mind, the vector that makes up each word has to be based on its context and not what it looks like. His example was we don't want 'car' to be similar to 'cat', even though they look similar. We instead want 'cat's vector to be similar to 'dog's vector (Vectoring Words (Word Embeddings) - Computerphile, 4:30-5:00). Another thing that was talked about in the video was kind of how neural networks accomplish this vectorization process. If the input layer is ten thousand neurons, then it condenses that information down to less neurons in the hidden layer. The network then uses probability to identify other similar vectors and it ends up outputting the information with another ten thousand neurons (Vectoring Words (Word Embeddings) - Computerphile, 7:00-8:00). The other resource we had was a research paper on model's that identify linguistic regularities in continuous word space. The paper talks about vector representations that were generated by a Recurrent Neural Network (RNN). It also goes on about how the vectors relate to one another and connect different words. They did have more success than the prior methods that attempted these things. Honestly, I was kind of confused by the paper, but the video made things make more sense.

2 Attention Grabbers

1. They trained the neural network on 320 million words!! That had to have taken forever. (Milotov, Yih, and Zweig, 4)

2. They achieved about 40% accuracy, which sounds really bad to me. However, it outperformed the old method, so it was a success. (Milotov, Yih, and Zweig, 5)
3. I thought the way he described using vectors to describe images was really interesting. I've done some work with that before, that made the representing words with vectors make slightly more sense. I was pretty confused after just reading the paper. (Vectoring Words (Word Embeddings) - Computerphile, 2:30-3:00)
4. The guy in the video talks about avoiding saying that words that are lexicographically similar (aka cat and car) are described as similar to the model. I can wrap my mind around how one would code identifying lexicographical similarities, but I'm trying to think about how you would code to recognize similar meanings of words and it hurts my brain. (Vectoring Words (Word Embeddings) - Computerphile, 4:30-5:00).
5. It's crazy that the vector for king minus the vector for man, plus the vector for woman results in the vector for Queen. The fact that not only similarity correlations exists, but you can do math with them too is just wild to me. (Vectoring Words (Word Embeddings) - Computerphile, 12:00)

3 Questions

1. What is the difference between a Recurrent Neural Network and a normal neural network?
2. What is the purpose/benefit of a model that can identify linguistic regularities?
3. How does compressing the data down to less neurons within the hidden layer, make it work better? Like why not have equal numbers of neurons in all the layers?
4. In the video he said it's the law that all word embedding models be trained on the same starter data, why is that?
5. How do models such as this avoid becoming sexist/racist? Because obviously certain things are associated with each gender and to some extent certain things relate to different races more than others.